



B.TECH. (CSE)

SEMESTER: IV SECTION: H

UE23CS252B – COMPUTER NETWORKS LABORATORY

PROJECT REPORT ON

NETWORK ATTACK & INTRUSION DETECTION SYSTEM

SUBMITTED BY

NAME SRN

1. Rithvik Rajesh Matta PES2UG23CS485
2. Rohan J Jogi PES2UG23CS489

JANUARY – MAY 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ELECTRONIC CITY CAMPUS,

BENGALURU – 560100, KARNATAKA, INDIA

Date: 24-04-2025

Problem Statement: Multiuser chat room with raw sockets

Abstract

This project is a multi-room chat server with encrypted communication via SSL. The system consists of a server and client components that enable secure text-based communication. Key features include:

- Encrypted connections using SSL certificates for secure communication
- Multiple chat rooms with the ability to create new rooms and switch between them
- User management with admin privileges (kick, ban, mute users)
- Emoji parsing to convert text codes into actual emoji characters
- Message logging to a SQLite database for chat history
- Command system with various utilities (/help, /join, /list, etc.)
- Proper error handling and connection management
- Thread-based client handling for concurrent connections
- System notifications for user events (joining/leaving)
- Simple database utility for viewing chat logs

The architecture follows a client-server model where each client connection is handled in a separate thread, allowing multiple users to communicate simultaneously across different chat rooms.

File Heirarchy

```
chatroom_project/  
├── __pycache__/  
├── database/  
│   └── chat_logs.db  
├── ssl_cert/  
│   ├── server.crt  
│   ├── server.csr  
│   └── server.key  
├── chatroom.py  
├── client.py  
└── db.py
```

└─ server.py

Code:

server.py

```
import socket
import threading
import ssl
import sqlite3
import time
import os
from chatroom import ChatRoom

ADMIN_USERS = ["admin"] # Example: hardcoded admin username
EMOJIS = {":smile:": "😊", ":sad:": "😞", ":laugh:": "😂",
":heart:": "❤️", ":thumbsup:": "👍"}

class ChatServer:
    def __init__(self, host='10.1.5.6', port=5555):
        self.host = host
        self.port = port
        self.server_socket = None
        self.running = False

        # Create SSL context
        self.context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
        self.context.load_cert_chain(certfile="ssl_cert/server.crt",
keyfile="ssl_cert/server.key")

        self.chatrooms = {}
        self.client_usernames = {} # socket -> username
        self.client_chatroom = {} # socket -> chatroom
        self.muted_users = set() # usernames
        self.banned_users = set() # usernames
```

```

        self.typing_users = set()    # currently typing users
        self.clients = []           # all connected clients

        # Make sure database directory exists
        os.makedirs("database", exist_ok=True)

        self.conn = sqlite3.connect("database/chat_logs.db",
check_same_thread=False)
        self.cursor = self.conn.cursor()
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS logs (
                username TEXT, room TEXT, message TEXT, timestamp DATETIME
DEFAULT CURRENT_TIMESTAMP
            )
        """)
        self.conn.commit()

    def initialize_socket(self):
        # Create a new socket
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)

        # Wrap with SSL
        self.server_socket = self.context.wrap_socket(self.server_socket,
server_side=True)

    def start_server(self):
        # Initialize default rooms
        for name in ["general", "tech", "fun"]:
            self.chatrooms[name] = ChatRoom(name)

        max_retries = 5
        retry_count = 0
        retry_delay = 2    # seconds

```

```

while retry_count < max_retries:
    try:
        self.initialize_socket()
        self.server_socket.bind((self.host, self.port))
        self.server_socket.listen(5)
        print(f"Server started on {self.host}:{self.port}")
        self.running = True
        break
    except socket.error as e:
        if e.errno == 98: # Address already in use
            print(f"Port {self.port} is already in use. Retrying in
{retry_delay} seconds...")
            retry_count += 1
            time.sleep(retry_delay)

            # Try with a different port if needed
            if retry_count >= 3:
                self.port += 1
                print(f"Trying with port {self.port}")
            else:
                print(f"Error binding socket: {e}")
                break

if not self.running:
    print("Failed to start server after multiple attempts")
    return

self.accept_connections()

def accept_connections(self):
    """Handle incoming connections in a loop"""
    while self.running:
        try:
            client_socket, addr = self.server_socket.accept()
            print(f"[LOG] Connection from {addr}")
            self.clients.append(client_socket)

```

```

        client_thread = threading.Thread(target=self.handle_client,
args=(client_socket,))
        client_thread.daemon = True
        client_thread.start()
    except Exception as e:
        if self.running: # Only show error if we're supposed to be
running
            print(f"Error accepting connection: {e}")

def handle_client(self, client_socket):
    """Handle a single client connection"""
    username = None
    try:
        client_socket.send(b"Enter your username: ")
        username = client_socket.recv(1024).decode().strip()

        if not username:
            client_socket.send(b"Invalid username. Disconnecting.\n")
            return

        if username in self.banned_users:
            client_socket.send(b"You are banned from this server.\n")
            return

        self.client_usernames[client_socket] = username

        welcome = f"\n[Welcome {username}]\nYou have joined the
chatroom!\nType /help for commands.\n"
        client_socket.send(welcome.encode())
        self.join_chatroom("general", client_socket)

        # Notify everyone that a new user has joined
        self.chatrooms["general"].broadcast(f"[System] {username} has
joined the server!", sender_socket=client_socket)

    while True:
        try:

```

```

        msg = client_socket.recv(1024).decode().strip()
        if not msg:
            break

        if msg.startswith("/"):
            if msg.lower() == "/quit":
                client_socket.send(b"Goodbye!\n")
                break
            self.handle_command(msg, client_socket)
        else:
            username = self.client_usernames[client_socket]
            if username in self.muted_users:
                client_socket.send(b"You are muted and cannot
send messages.\n")
                continue

            room = self.client_chatroom[client_socket].name
            message = self.parse_emoji(f"[User: {username}]
{msg}")

            self.client_chatroom[client_socket].broadcast(message,
sender_socket=client_socket)

            # Log the message to database
            self.cursor.execute("INSERT INTO logs (username,
room, message) VALUES (?, ?, ?)",
                                (username, room, msg)) # Store
original message, not the emoji-parsed one
            self.conn.commit()
        except ConnectionResetError:
            print(f"[LOG] Connection reset by {username}")
            break
        except Exception as e:
            print(f"Error receiving message from {username}: {e}")
            break
    except Exception as e:
        print(f"Error handling client: {e}")

```

```

        finally:
            self.disconnect_client(client_socket)

    def handle_command(self, msg, client_socket):
        cmd = msg.split(" ")
        username = self.client_usernames[client_socket]
        current_room = self.client_chatroom.get(client_socket)

        if cmd[0] == "/help":
            help_msg = """
Available commands:
/help                Show this message
/quit                Exit chat
/join <room>         Join existing room
/create <room>       Create and join a new room
/list                List users in your room
/rooms               List all available rooms
/kick <username>     (Admin) Kick a user
/ban <username>      (Admin) Ban a user
/mute <username>     (Admin) Mute a user
/unmute <username>   (Admin) Unmute a user
            """
            client_socket.send(help_msg.encode())

        elif cmd[0] == "/quit":
            # This will be handled in the main loop
            pass

        elif cmd[0] == "/create" and len(cmd) == 2:
            roomname = cmd[1]
            if roomname not in self.chatrooms:
                self.chatrooms[roomname] = ChatRoom(roomname)
                client_socket.send(f"Room '{roomname}'
created.\n".encode())
                self.join_chatroom(roomname, client_socket)

        elif cmd[0] == "/join" and len(cmd) == 2:

```



```
        roomname = cmd[1]
        if roomname in self.chatrooms:
            self.join_chatroom(roomname, client_socket)
        else:
            client_socket.send(f"Room '{roomname}' does not exist. Use
/create {roomname} to create it.\n".encode())

    elif cmd[0] == "/list":
        if current_room:
            users = current_room.list_users()
            client_socket.send(f"Users in {current_room.name}: {'',
'.join(users)}\n".encode())
        else:
            client_socket.send(b"You are not in any room.\n")

    elif cmd[0] == "/rooms":
        rooms = ", ".join(self.chatrooms.keys())
        client_socket.send(f"Available rooms: {rooms}\n".encode())

    elif cmd[0] == "/kick" and len(cmd) == 2:
        if username in ADMIN_USERS:
            target = cmd[1]
            self.kick_user(target, reason="Kicked by admin")
            client_socket.send(f"User {target} kicked.\n".encode())
        else:
            client_socket.send(b"Permission denied. Admin privileges
required.\n")

    elif cmd[0] == "/ban" and len(cmd) == 2:
        if username in ADMIN_USERS:
            target = cmd[1]
            self.ban_user(target)
            client_socket.send(f"User {target} banned.\n".encode())
        else:
            client_socket.send(b"Permission denied. Admin privileges
required.\n")
```

```

elif cmd[0] == "/mute" and len(cmd) == 2:
    if username in ADMIN_USERS:
        self.muted_users.add(cmd[1])
        client_socket.send(f"User {cmd[1]} muted.\n".encode())
    else:
        client_socket.send(b"Permission denied. Admin privileges
required.\n")

elif cmd[0] == "/unmute" and len(cmd) == 2:
    if username in ADMIN_USERS:
        if cmd[1] in self.muted_users:
            self.muted_users.discard(cmd[1])
            client_socket.send(f"User {cmd[1]}
unmuted.\n".encode())
        else:
            client_socket.send(f"User {cmd[1]} is not
muted.\n".encode())
        else:
            client_socket.send(b"Permission denied. Admin privileges
required.\n")

    else:
        client_socket.send(b"Invalid command. Type /help for available
commands.\n")

def kick_user(self, target_username, reason="You have been kicked from
the server."):
    """Kick a user from the server"""
    for sock, name in list(self.client_usernames.items()):
        if name == target_username:
            sock.send(f"{reason}\n".encode())
            self.disconnect_client(sock)
            return True
    return False

def ban_user(self, target_username):
    """Ban a user from the server"""

```

```
self.banned_users.add(target_username)
return self.kick_user(target_username, reason="You have been banned
from the server.")

def join_chatroom(self, roomname, client_socket):
    """Join a user to a chatroom"""
    current = self.client_chatroom.get(client_socket)
    username = self.client_usernames.get(client_socket, "Unknown")

    if current:
        current.broadcast(f"[System] {username} left room
'{current.name}'.", sender_socket=None)
        current.remove_client(client_socket)

    self.chatrooms[roomname].add_client(client_socket)
    self.client_chatroom[client_socket] = self.chatrooms[roomname]
    client_socket.send(f"You joined room '{roomname}'.\n".encode())

def disconnect_client(self, client_socket):
    """Handle client disconnection"""
    try:
        username = self.client_usernames.get(client_socket, "Unknown")
        print(f"[LOG] {username} disconnected")

        # Remove from chatroom
        room = self.client_chatroom.get(client_socket)
        if room:
            room.remove_client(client_socket)
            room.broadcast(f"[System] {username} left the chat.")

        # Clean up dictionaries
        self.client_usernames.pop(client_socket, None)
        self.client_chatroom.pop(client_socket, None)

        if client_socket in self.clients:
            self.clients.remove(client_socket)
```

```

        # Close socket
        client_socket.close()
    except Exception as e:
        print(f"Error during client disconnect: {e}")

def parse_emoji(self, message):
    """Replace emoji codes with actual emojis"""
    for emoji_code, emoji in EMOJIS.items():
        message = message.replace(emoji_code, emoji)
    return message

def shutdown(self):
    """Shut down the server properly"""
    self.running = False

    # Notify and disconnect all clients
    for client in list(self.clients):
        try:
            client.send(b"Server is shutting down. Goodbye!\n")
            client.close()
        except:
            pass

    # Close server socket
    if self.server_socket:
        self.server_socket.close()

    # Close database connection
    self.conn.close()
    print("Server shut down.")

if __name__ == "__main__":
    try:
        server = ChatServer()
        server.start_server()
    except KeyboardInterrupt:
        print("\nShutting down server...")

```

```
server.shutdown()
except Exception as e:
    print(f"Fatal error: {e}")
```

client.py

```
import socket
import ssl
import threading
import time
import sys

def receive_messages(client_socket):
    """Continuously receive and display messages from the server"""
    try:
        buffer = ""
        while True:
            try:
                data = client_socket.recv(1024).decode('utf-8')
                if not data:
                    print("Server closed the connection.")
                    break

                buffer += data

                # Process complete messages (in case we get multiple at
once)
                lines = buffer.split('\n')
                # Keep the last line if it's incomplete
                buffer = lines.pop() if lines and not data.endswith('\n')
            else ""

            # Print each complete line
            for line in lines:
                if line.strip(): # Only print non-empty lines
                    print(line)
```

```

        except ssl.SSLError as e:
            if e.errno == ssl.SSL_ERROR_WANT_READ:
                # Non-blocking SSL socket needs more data, just
continue
                continue
            else:
                print(f"SSL Error: {e}")
                break
        except Exception as e:
            print(f"Error in receive: {e}")
            break
    except Exception as e:
        print(f"Receive thread error: {e}")
    finally:
        print("Disconnected from server")

def start_client():
    """Connect to the server and handle user input"""
    client_socket = None
    try:
        # Create socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Connect first, then wrap with SSL
        print("Connecting to server...")
        client_socket.connect(('10.1.5.6', 5555))

        # Set up SSL context
        context = ssl.create_default_context()
        context.check_hostname = False
        context.verify_mode = ssl.CERT_NONE # For development only, not
secure for production

        # Wrap the socket with SSL
        client_socket = context.wrap_socket(client_socket,
server_hostname="10.1.5.6")
        print("Connected to server.")

```

```

    # Handle initial username prompt
    username_prompt = client_socket.recv(1024).decode('utf-8')
    username = input(username_prompt)
    client_socket.send(username.encode('utf-8'))

    # Start the receive thread
    receive_thread = threading.Thread(target=receive_messages,
args=(client_socket,))
    receive_thread.daemon = True
    receive_thread.start()

    print("\nStart typing messages (type /quit to exit):")
    while True:
        try:
            message = input()
            if message.lower() == "/quit":
                client_socket.send(message.encode('utf-8'))
                print("Goodbye!")
                time.sleep(1)  # Give time for server to process quit
command
                break
            else:
                client_socket.send(message.encode('utf-8'))
        except BrokenPipeError:
            print("Connection to server lost.")
            break
        except ConnectionResetError:
            print("Connection reset by server.")
            break
        except Exception as e:
            print(f"Error sending message: {e}")
            break

    except ConnectionRefusedError:
        print("Connection refused. Is the server running?")
    except Exception as e:

```

```

        print(f"Error: {e}")
    finally:
        if client_socket:
            try:
                client_socket.shutdown(socket.SHUT_RDWR)
                client_socket.close()
            except:
                pass
        print("Client terminated.")

if __name__ == "__main__":
    start_client()

```

chatroom.py

```

class ChatRoom:
    def __init__(self, name):
        self.name = name
        self.clients = []

    def add_client(self, client_socket):
        """Add a client to the chatroom"""
        if client_socket not in self.clients:
            self.clients.append(client_socket)

    def remove_client(self, client_socket):
        """Remove a client from the chatroom"""
        if client_socket in self.clients:
            self.clients.remove(client_socket)

    def broadcast(self, message, sender_socket=None):
        """Send a message to all clients in the room except the sender"""
        # Ensure the message ends with a newline for proper display
        if not message.endswith('\n'):
            message += '\n'

        message_bytes = message.encode('utf-8')

```



```

        # Make a copy of the clients list to avoid modification during
iteration
        clients_copy = self.clients.copy()

        for client in clients_copy:
            # Don't send back to the sender
            if client != sender_socket:
                try:
                    client.send(message_bytes)
                except Exception as e:
                    print(f"Error broadcasting to client: {e}")
                    # Don't remove here, let the server handle disconnects

    def list_users(self):
        """Return a list of usernames in the room"""
        # This needs to be implemented by the server since it has the
username mapping
        # We'll return client socket identifiers here, and the server will
translate
        return [str(client.getpeername()) for client in self.clients]

```

db.py

```

import sqlite3

# Connect to the database
conn = sqlite3.connect("database/chat_logs.db")
cursor = conn.cursor()

# Query all data from logs table
cursor.execute("SELECT * FROM logs")
rows = cursor.fetchall()

# Print the data
print("Username | Room | Message | Timestamp")
print("-" * 60)

```

```
for row in rows:
    print(f"{row[0]} | {row[1]} | {row[2]} | {row[3]}")

# Close connection
conn.close()
```

Output screenshots:

Server

```
rithvikmatta@penguin:~$ cd CN-project/
rithvikmatta@penguin:~/CN-project$ cd chatroom_project/
rithvikmatta@penguin:~/CN-project/chatroom_project$ ls
chatroom.py  client2.py  client.py  database  db.py  __pycache__  server.py  ssl_cert
rithvikmatta@penguin:~/CN-project/chatroom_project$ python3 server.py
Server started on localhost:5555
[LOG] Connection from ('127.0.0.1', 51926)
[LOG] Connection from ('127.0.0.1', 35280)
[LOG] rithvik disconnected
[LOG] rohan disconnected
^C
Shutting down server...
Server shut down.
rithvikmatta@penguin:~/CN-project/chatroom_project$
```

Client 1

```
rithvikmatta@penguin:~/CN-project/chatroom_projects$ python3 client.py
Connecting to server...
Connected to server.
Enter your username: rohan

Start typing messages (type /quit to exit):
[Welcome rohan]
You have joined the chatroom!
Type /help for commands.
You joined room 'general'.
hey
[User: rithvik] hi rohan, i'm rithvik
hey rithvik
[User: rithvik] wanna hop on the tech chatroom ?
sure
/join tech
[System] rohan left room 'general'.
You joined room 'tech'.
[User: rithvik] hii
hey
[User: rithvik] did u notice the commands, this interface is so good 😊
yeahhh
apparently admin has chatroom-handling privileges
[User: rithvik] ohh yeahh
[User: rithvik] he can kick, ban,mute,unmute ppl
ohh righttt
[User: rithvik] hey, i've gtg bye
bye then
[System] rithvik left the chat.
/quit
Goodbye!
Goodbye!
Server closed the connection.
Disconnected from server
Client terminated.
rithvikmatta@penguin:~/CN-project/chatroom_projects$
```

Client 2

```

rithvikmatta@penguin:~/CN-project/chatroom_project$ python3 client.py
Connecting to server...
Connected to server.
Enter your username: rithvik

Start typing messages (type /quit to exit):
[Welcome rithvik]
You have joined the chatroom!
Type /help for commands.
You joined room 'general'.
[System] rohan has joined the server!
[User: rohan] hey
hi rohan, i'm rithvik
[User: rohan] hey rithvik
wanna hop on the tech chatroom ?
[User: rohan] sure
[System] rohan left room 'general'.
/help
Available commands:
/help          Show this message
/quit          Exit chat
/join <room>    Join existing room
/create <room> Create and join a new room
/list          List users in your room
/rooms         List all available rooms
/kick <username> (Admin) Kick a user
/ban <username> (Admin) Ban a user
/mute <username> (Admin) Mute a user
/unmute <username> (Admin) Unmute a user
/tech          Invalid command. Type /help for available commands.

/join tech
[System] rithvik left room 'general'.
You joined room 'tech'.
hii
[User: rohan] hey
did u notice the commands, this interface is so good :smile:
[User: rohan] yeahhh
[User: rohan] apparently admin has chatroom-handling priviledges
ohh yeahh
he can kick, ban,mute,unmute ppl
[User: rohan] ohh righttt
hey, i've gtg bye
[User: rohan] bye then
/quit
Goodbye!

```

Database (containing chatlog)

```

rithvikmatta@penguin:~/CN-project/chatroom_project$ python3 db.py
Username | Room | Message | Timestamp
-----
fe | general | wrte | 2025-04-20 06:09:15
fe | general | erferf | 2025-04-20 06:09:17
egge | general | hello | 2025-04-20 06:10:15
egge | general | my name is egge | 2025-04-20 06:10:20
fe | general | hi egge | 2025-04-20 06:10:24
fe | general | wassup | 2025-04-20 06:10:28
fe | general | :smile: | 2025-04-20 06:10:38
fe | general | :smile: | 2025-04-20 06:10:44
fe | tech | fewfew | 2025-04-20 06:12:21
egge | general | join fun | 2025-04-20 06:12:28
egge | general | join tech | 2025-04-20 06:12:37
fe | tech | vreq | 2025-04-20 06:12:41
egge | general | erferfe | 2025-04-20 06:12:43
egge | general | frqe | 2025-04-20 06:12:48
fe | tech | veef | 2025-04-20 06:13:08
egge | tech | ferrerf | 2025-04-20 06:13:10
admin | general | frefe | 2025-04-20 06:14:04
admin | tech | frfw | 2025-04-20 06:14:20
admin | tech | fefe | 2025-04-20 06:14:39
admin | tech | ferfer | 2025-04-20 06:14:41
fe | tech | grtew | 2025-04-20 06:14:43
fe | tech | fenferwf | 2025-04-20 06:14:45
admin | tech | hello | 2025-04-20 06:21:09
fregf | general | :smile: | 2025-04-20 06:22:17
ded | general | gegerwg | 2025-04-20 08:24:15
ded | tech | gewrtgerg | 2025-04-20 08:24:20
admin | tech | gtegegt | 2025-04-20 08:24:26
admin | tech | getgetgetg | 2025-04-20 08:24:30
admin | tech | gtget | 2025-04-20 08:24:33
admin | tech | :smile: | 2025-04-20 08:24:40
rtk | fun | gew | 2025-04-21 04:43:30
rtk | fun | gerf | 2025-04-21 04:43:47
rohan | general | hey | 2025-04-24 11:01:58
rithvik | general | hi rohan, i'm rithvik | 2025-04-24 11:02:06
rohan | general | hey rithvik | 2025-04-24 11:02:10
rithvik | general | wanna hop on the tech chatroom ? | 2025-04-24 11:02:20
rohan | general | sure | 2025-04-24 11:02:23
rithvik | tech | hii | 2025-04-24 11:02:41
rohan | tech | hey | 2025-04-24 11:02:43
rithvik | tech | did u notice the commands, this interface is so good :smile: | 2025-04-24 11:03:00
rohan | tech | yeahhh | 2025-04-24 11:03:06
rohan | tech | apparently admin has chatroom-handling priviledges | 2025-04-24 11:03:25
rithvik | tech | ohh yeahh | 2025-04-24 11:03:30

```